
APMTH 50 Problem Set 2

Due on Friday, March 3, by 5pm

Collaboration policy

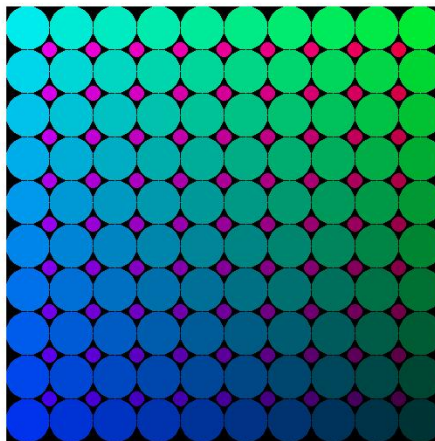
Collaboration on homework and labs is encouraged, but each student must write solutions in their own words, and not copy them from anyone else. Please write the name(s) of your collaborator(s) at the top of the homework.

Submission guidelines

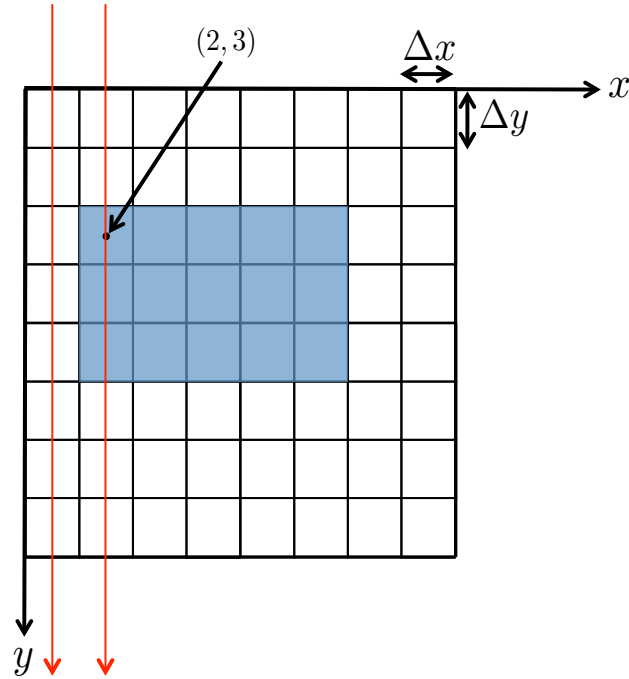
Please upload your homework as a single Word or pdf document. Unless stated otherwise, you do not need to include your code in your submission, but you should include all output (such as figures/plots) from your code.

Problems

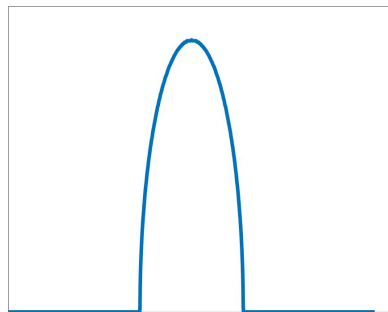
1. Describe an inverse problem application that we have not talked about in class. Write a few sentences about the problem, clearly stating the known and unknown quantities of the inverse problem. You can come up with the problem on your own, or you can find a problem online or in other source. Please cite all human and non-human sources.
2. Write a Matlab program that produces the 500×500 pixel color image below. Your solution should include both your code and the output image from the code.



3. The blue rectangle and white background have respective attenuation coefficients of 1 and 0 in units of inverse centimeters. The dimension of each pixel is $\Delta x = \Delta y = 1$ cm.



- (a) By hand, calculate the projections for rays that form 0, 45, and 90 degree angles with respect to the y -axis, and sketch the projections in each case. Be sure to calculate eight values for each projection. For example, for the 0 degree projection, your rays should go through the center of each pixel in each column of the image matrix. Two of the eight rays are marked by red lines in the figure.
- (b) Without using Matlab's Radon transform function, write a program that creates the image, calculates the projections from part (a), and plots the projections.
4. The Radon transform of an object is given below. Suppose the Radon transform of this object is the same for every angle θ ; that is, taking the Radon transform at any angle gives the projection below. What could the object be? Sketch the object. Then generate the object in Matlab and calculate and plot the Radon transform for a few angles. Please include your Matlab code in your submission.



-
5. In last week's lab, you used the command `sinogram = radon(image, angles)` to calculate the Radon transform of `image` for a set of `angles`. Recall that each column of `sinogram` corresponded to the Radon transform of the object at a specified angle. In this last problem we'll qualitatively explore *backprojection*, one of the main methods for inverting the Radon transform and reconstructing an image for CT.

Write a Matlab code to:

- (a) Create a 1000×1000 pixel black image with a white circle of radius 100 pixels at the center of the image.
- (b) Calculate the Radon transform with `sinogram = radon(image, 0:dtheta:180)` for `dtheta = [90, 60, 45, 30, 15]`. For each value of `dtheta`, invert the transform to obtain a reconstructed version of the original image. Please do this with a loop.
- (c) On one figure, create six subfigures: the original image from part (a) and the five reconstructed images from part (b). Label each subfigure with the corresponding value of `dtheta`.

Run the code and stare at the figure! Notice that the reconstructed image gets better as more projections are used in the reconstruction. Along with the circle at the center of the image, you should see stripes in the image at various angles. These stripes are roughly the result of smearing each *projection back* through the image along the rays in which it was originally acquired, so that all pixels along a particular ray are set to the same value. If you click on the reconstructed image though, you'll notice that the pixels don't have the same values along the stripes. This is because Matlab uses a more sophisticated type of backprojection to reconstruct the image, called *filtered backprojection*, which is the most commonly used algorithm for CT systems.

- (d) Replace the image in part (a) by your house photo. To do this, load your house photo, make it black-and-white, and convert it from `uint8` to `double`. Then re-run your code. Are you satisfied with the reconstructed images? Modify `dtheta` until you obtain a visually satisfactory reconstructed image.